

Supplementary Materials

This document provides additional formulation details, numerical results, sensitivity analyses, and clarifications for **Diff-GNN**. In particular, it addresses the exactness and scalability of the MILP comparison, runtime fairness, surrogate fidelity, decoding choices, and the scope of the cost and platform models. The main paper evaluates real TOSA benchmark task-graph topologies, while measured platform-specific execution, communication, and area costs, as well as deployment on a real system, remain future work.

1 Scheduling Constrained Partitioning Problem with MILP

1.1 Problem Definition

We are given a graph $G = (V, E)$ with n nodes and m edges. Each node has weights h_i and s_i to denote hardware and software execution times. Also, each node has an associated hardware area cost a_i . The edges denote communication between the nodes and have associated communication cost c_{ij} for the edge e connecting nodes i and j . The communication delay is taken into account only when the adjacent nodes belong to different partitions. The aim is to partition the nodes V into non-overlapping sets V_S and V_H denoting software and hardware nodes respectively; $V = V_H \cup V_S$ and $V_H \cap V_S = \emptyset$. We denote the set of edges between the two sets by E_P . Additionally, we need to ensure that software nodes are executed sequentially, while hardware nodes can execute in parallel if their dependent nodes are executed. Our objective is to minimize the overall execution time given a limited available area for hardware nodes.

1.2 Optimization Formulation

Decision variables. We define binary variable $x_i = 0$ if node i is assigned to hardware partition, $x_i = 1$ if assigned to software partition. We also define binary variable z_{ij} for each edge $e := (i, j) \in E$ connecting nodes i and j . We denote $z_{ij} = 1$ if the nodes i and j belong to different partitions. Let t_i and f_i denote the start and end time of the node i . The total execution time of the task graph is denoted by T . We also define binary variable y_{ij} for each pair of nodes i, j , $i \neq j$. This binary variable is used to ensure that if the pair of nodes i, j are assigned to software partition, they need to be executed sequentially; i.e., if $y_{ij} = 1$, then node i precedes node j , and for $y_{ij} = 0$, the opposite happens. However, we need to ensure that this sequential constraint is activated for only a pair of software nodes.

Hardware area constraint. The overall hardware area is constrained by the maximum allowable hardware area $A_{\max}$.

$$\sum_{i=1}^n a_i (1 - x_i) \leq A_{\max} \quad (1)$$

Binary edge variable. The binary variable z_{ij} and the partition variable x_i, x_j corresponding to

the adjacent nodes are related by

$$z_{ij} \geq x_i - x_j, \quad \forall (i, j) \in E \quad (2a)$$

$$z_{ij} \geq x_j - x_i, \quad \forall (i, j) \in E \quad (2b)$$

$$z_{ij} \leq x_i + x_j, \quad \forall (i, j) \in E \quad (2c)$$

$$z_{ij} \leq 2 - x_i - x_j, \quad \forall (i, j) \in E \quad (2d)$$

Execution time constraints. The execution time of each node is denoted

$$T \geq f_i, \quad \forall i \in V \quad (3a)$$

$$f_i = t_i + h_i(1 - x_i) + s_i x_i, \quad \forall i \in V \quad (3b)$$

$$f_i + c_{ij} z_{ij} \leq t_j, \quad \forall (i, j) \in E \quad (3c)$$

Sequential execution of software nodes. We define the following pair of constraints with M being a large positive number.

$$f_i \leq t_j + M(1 - y_{ij}) + M(2 - x_i - x_j), \quad \forall i, j \in V, i \neq j \quad (4a)$$

$$f_j \leq t_i + M y_{ij} + M(2 - x_i - x_j), \quad \forall i, j \in V, i \neq j \quad (4b)$$

Note that, the constraints are activated only when $x_i = x_j = 1$ or the nodes i, j are both software nodes, when the term $2 - x_i - x_j = 0$. For other cases, the constraints are not active since M is sufficiently large. Once the constraints are activated, the value of y_{ij} denotes the precedence of the software node operations.

The overall optimization problem is

$$\text{minimize } T \quad (5a)$$

$$\text{subject to } (1), (2), (3), (4) \quad (5b)$$

1.3 Experimental comparison with order-aware MILP

Note that the pairwise sequential execution constraints for software nodes scale quadratically, making the MILP formulation computationally expensive for large task graphs. It requires $O(n^2)$ time to order binaries and constraints, making exact optimization impractical beyond small graphs. Table 1 illustrates this scaling behavior. The order-aware MILP is exact on the smallest instance, but its runtime grows rapidly, and it fails to return a solution for the 68-node graph within two hours. In contrast, Diff-GNN returns competitive schedules within approximately one minute.

Table 1: Comparison with the order-aware MILP. NA indicates that the solver did not return a solution within the time limit.

Graph	n	Order binaries	Optimal	Diff-GNN	MILP time (s)	Diff-GNN time (s)
paper_fig3_11node	11	55	29.000	29.000	0.28	7.20
squeezenet_30	30	435	248.915	267.915	1853.47	30.82
squeezenet_40	40	712	362.742	404.647	6840.00	50.01
anomaly_detection_tosa	68	2278	NA	1373.646	> 7200	56.824

These results motivate using the scalable placement-only MILP in the main experiments while clearly labeling it as optimistic. They also show that **Diff-GNN** recovers the optimum on the 11-node instance and remains close to the exact solution on larger instances for which the MILP is still tractable.

2 Runtime-Matched Comparison and Scalability

To separate solution quality from optimization budget, we compare all methods under the same 100-second wall-clock budget. As shown in Table 2, **Diff-GNN** achieves the lowest mean makespan among all evaluated methods. Thus, its improvement is not explained by a larger runtime allocation.

Table 2: Runtime-matched makespan comparison under a 100-second budget.

Method	Makespan	Method	Makespan
Diff-GNN	1635.466±52.523	SHADE	2862.378±513.612
ESA	1975.949±501.683	DBPSO	3095.608±87.251
GL25	2137.484±111.681	Greedy	3110.597±757.747
CCPSO	2287.959±525.583	Random	3149.357±168.282
GCPS	2825.011±618.821	JADE	3150.200±289.347
CLPSO	2858.078±184.411	PSO	3274.278±213.959

The scaling results further demonstrate the benefit of the GNN-based formulation. On **mobile_net_tosa**, **Diff-GNN** reduces makespan by 20.75% relative to the best metaheuristic baseline. On the 1,000-node synthetic graph, it also outperforms all evaluated baselines. The 10,000-node stress test requires approximately two hours, whereas exact scheduling-aware MILP is already impractical at tens of nodes. Runtime can be controlled through the number of training epochs, the refinement budget, and sparse top- k ordering. Moreover, **Diff-GNN** can exploit GPU-parallel tensor operations, whereas most classical metaheuristics rely on iterative CPU-based search.

3 Detailed Numerical Results

Table 3 reports the numerical values underlying the plots in the main paper. The placement-only MILP is shown only as an optimistic reference and is not treated as a feasible full-problem solution. Among methods that produce realizable schedules, **Diff-GNN** achieves the best mean makespan on all eight graphs.

Table 3: Detailed makespan comparison. Values are mean±standard deviation. MILP is an optimistic placement-only reference.

Graph	MILP	Diff-GNN	GCPS	Greedy	PSO	Random	CCPSO	CLPSO	DBPSO	ESA	SHADE	GL25
anomaly.	1499.26±339	1668.44±301	3101.45±525	3335.92±516	2945.88±310	3162.85±201	3194.85±175	3232.89±280	3018.63±325	3205.61±345	2851.09±301	1680.53±284
image.	3200.24±369	3281.02±289	6519.84±647	7558.28±817	5975.61±333	6293.95±418	6293.95±418	6293.95±418	6293.95±418	6293.95±418	6293.95±418	3950.91±314
keyword.	2913.31±272	3298.76±315	6681.34±589	7209.40±548	5898.59±381	6205.82±407	6205.82±407	6205.82±407	6205.82±407	6205.82±407	6205.82±407	4063.25±536
mobile.	NA	33261.56±13487	44500.37±2201	53250.63±2751	41973.24±786	41973.24±786	41973.24±786	41973.24±786	41973.24±786	41973.24±786	41973.24±786	41973.24±786
rez_net	9218.73±538	14296.66±1444	17582.97±1165	21803.94±1312	17813.67±486	18287.07±491	18287.07±491	18287.07±491	18287.07±491	18287.07±491	18287.07±491	14652.41±912
squeeze.	4241.80±402	5462.24±517	9882.35±799	11534.32±915	8673.90±432	8958.48±458	8958.48±458	8958.48±458	8958.48±458	8958.48±458	8958.48±458	6772.41±375
squeeze_1k.	NA	40522.50±3790	44356.35±1676	50473.41±1024	44051.20±1019	50473.41±1024	50473.41±1024	50473.41±1024	50473.41±1024	50326.15±1238	50473.41±1024	45114.04±962
visual.	5662.42±612	5657.40±567	11617.12±632	12367.92±1057	10508.68±368	10582.41±450	10582.41±450	10582.41±450	10582.41±450	10582.41±450	10582.41±450	8030.97±470

4 Hyperparameter Sensitivity

Table 4 evaluates the area penalty, relaxation temperatures, number of training epochs, and refinement budget on `anomaly_detection_tosa` with area ratio $\alpha = 0.5$. All tested configurations remain feasible and competitive. The larger area penalty produces the best result, while training epochs and refinement iterations do not yield monotonic improvements. This indicates that additional computation does not necessarily improve the decoded schedule after a strong candidate has been identified.

Table 4: Hyperparameter sensitivity on `anomaly_detection_tosa`.

Variant	Area coeff.	Temp./sharpness	Epochs	Refine iters.	Makespan	Runtime (s)
area_penalty_low	10^8	0.20/0.30/8.0	300	160	1419.955	47.277
area_penalty_high	10^{10}	0.20/0.30/8.0	300	160	1373.646	56.824
temperature_smooth	10^9	0.40/0.50/4.0	300	160	1485.859	49.465
temperature_sharp	10^9	0.10/0.20/12.0	300	160	1499.256	49.444
epochs_low	10^9	0.20/0.30/8.0	150	160	1408.289	33.482
epochs_high	10^9	0.20/0.30/8.0	450	160	1508.119	72.932
refinement_low	10^9	0.20/0.30/8.0	300	80	1615.586	45.344
refinement_high	10^9	0.20/0.30/8.0	300	240	1615.586	49.910

4.1 Sparse Top- k Ordering

The dense software-order matrix has quadratic cost. We therefore evaluate sparse top- k ordering, in which each task retains only its strongest candidate precedence relations. Table 5 shows that moderate sparsification preserves the ordering benefit while controlling runtime. Performance is not monotonic in k , but $k = 16$ obtains the best makespan in this experiment.

Table 5: Sensitivity to sparse top- k ordering.

k	Feasible	Makespan	Runtime (s)
8	Yes	1615.586	46.715
16	Yes	1435.522	63.919
32	Yes	1508.119	85.046
64	Yes	1508.119	85.882

5 Fidelity of the Differentiable Scheduler

The differentiable scheduler is designed as a smooth approximation to the discrete list scheduler. During training, the soft makespan closely tracks the discrete makespan under both static and learned priorities, showing that the surrogate provides a meaningful optimization signal. In the optimization trace presented in the main paper, approximately 62.5% of the total makespan improvement is obtained during differentiable GNN optimization, while post-processing contributes the remaining 37.5%. Thus, post-processing improves an already strong GNN solution rather than accounting for the full gain.

The learned software priorities also align closely with static critical-path priorities on the illustrated task graph. This agreement is graph-dependent and is not assumed to hold universally;

the learned ordering head remains useful because it can adapt the software schedule to the jointly optimized placement.

6 Decoding, Feasibility Repair, and Refinement

Diff-GNN does not depend on a single fixed threshold for decoding relaxed placement probabilities. Instead, it sweeps multiple thresholds, repairs each candidate to satisfy the hardware-area budget, evaluates the resulting discrete schedule, and selects the candidate with the lowest makespan. Consequently, the best threshold may differ across problem instances.

The refinement stage accepts only moves that improve the discrete objective. It often produces a large initial reduction in makespan, followed by diminishing gains. This behavior explains why increasing the refinement budget does not always improve the final result and provides a direct quality–runtime trade-off.

7 Evaluation Scope and Cost Realism

The experiments use real TOSA benchmark task-graph topologies together with the sampled cost model of Arato [1]. The same costs, area budgets, and scheduling evaluator are used for all methods, enabling controlled and reproducible comparisons across diverse hardware-speedup, area, and communication regimes.

The contribution of this work is the optimization framework rather than a platform-specific profiling flow. **Diff-GNN** accepts software execution time, hardware execution time, area, and communication costs as inputs regardless of their source. In a deployed CPU–FPGA or CPU–accelerator system, these inputs could be obtained from software profiling, HLS/RTL synthesis, static timing analysis, and measured data-transfer latency. FPGA area can be represented using LUTs, flip-flops, DSPs, and BRAMs, while ASIC area can be obtained from synthesized cell area. Deployment and end-to-end assessment using such measured costs are left for future work.

8 Communication and Platform Assumptions

The current communication model charges a transfer cost when dependent tasks are placed on different platforms. It does not explicitly model shared-bus contention, bandwidth limits, memory-system bottlenecks, or arbitration. These effects can be incorporated by replacing the scalar edge costs with resource-dependent transfer costs or by adding differentiable capacity and contention constraints to the scheduler.

The current formulation assumes one sequential software processor and parallel hardware tasks subject to DAG dependencies. This setting is common in HW/SW partitioning and isolates the joint placement-and-ordering problem studied here. Richer heterogeneous systems can be modeled by introducing resource-specific assignment probabilities, per-resource ordering variables, and differentiable capacity constraints. These extensions are promising directions for future work.

9 Coverage of Gradient-Based Baselines

GCPS provides a relevant gradient-based placement baseline and is included in the evaluation. However, it does not optimize an order-aware differentiable scheduling objective. The main distinction of **Diff-GNN** is that placement, communication, and software-task ordering are optimized

jointly through a differentiable scheduler. The placement-only Diff-GNN ablation further isolates the contribution of the ordering head.

10 Reproducibility

Implementations and configurations are available at <https://github.com/anonymousauthors001/Diff-GNN>. All methods are evaluated using the same task graphs, sampled cost instances, area budgets, feasibility repair, and final discrete makespan evaluator. Baseline implementations and parameter settings are included to support reproduction of the reported results.

References

- [1] ARATÓ, P., MANN, Z. Á., AND ORBÁN, A. Algorithmic aspects of hardware/software partitioning. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 10, 1 (2005), 136–156.